

Booking Service Design Document

Purpose: Manage the complete booking lifecycle, calculate pricing, generate tickets, and publish booking events. Booking is the source of truth for booking state.

Responsibilities

Create Pending Booking
Calculate pricing
Maintain booking lifecycle
Generate Ticket
Persist Transactional Outbox
Publish Booking events

Owned Entities

Booking, BookingSeat, Ticket, Outbox

Source of Truth

Booking lifecycle, booking pricing snapshot, ticket generation.

Architecture

Gateway -> Booking -> Saga -> Inventory(Hold Seats) -> Payment -> Booking
Confirmed -> Outbox -> Kafka -> Notification

ER Diagram

```
Booking ----- booking_id (PK) user_id show_id status
subtotal discount_amount coupon_discount tax_amount total_amount
created_at updated_at version 1 | * BookingSeat -----
booking_seat_id (PK) booking_id (FK) seat_id seat_type base_price
final_price 1 | 1 Ticket ----- ticket_id (PK)
booking_id (FK) ticket_number issued_at qr_code status Outbox
----- event_id aggregate_id aggregate_type event_type
payload status retry_count created_at published_at
```

Database Indexes

Booking(user_id); Booking(show_id); Booking(status); BookingSeat(booking_id);
Ticket(ticket_number UNIQUE); Outbox(status,created_at)

Published Events

BookingCreated, BookingConfirmed, BookingCancelled

Consumed Events

SeatHeld, SeatHoldFailed, PaymentSucceeded, PaymentFailed

Business Rules

Booking starts in PENDING state. Pricing is calculated during booking. Seat prices are stored as snapshots. Ticket is generated only after payment succeeds. Booking is cancelled if seat hold or payment fails.

Booking State Machine

PENDING -> PAYMENT_IN_PROGRESS -> CONFIRMED OR CANCELLED

Transactional Outbox

Booking transaction writes both Booking and Outbox records atomically. Background publisher sends events to Kafka with retry support.

Scalability & Reliability

Idempotent consumers, optimistic locking where required, retry with backoff, DLQ support, metrics and tracing.